

DRIVER EARNINGS NAVIGATOR

Machine Learning Relocation Recommendations for NYC Uber Drivers

A full-stack app that ranks nearby TLC zones using historical Uber trip patterns, estimated travel time, and adjusted earning exposure.

3.19M

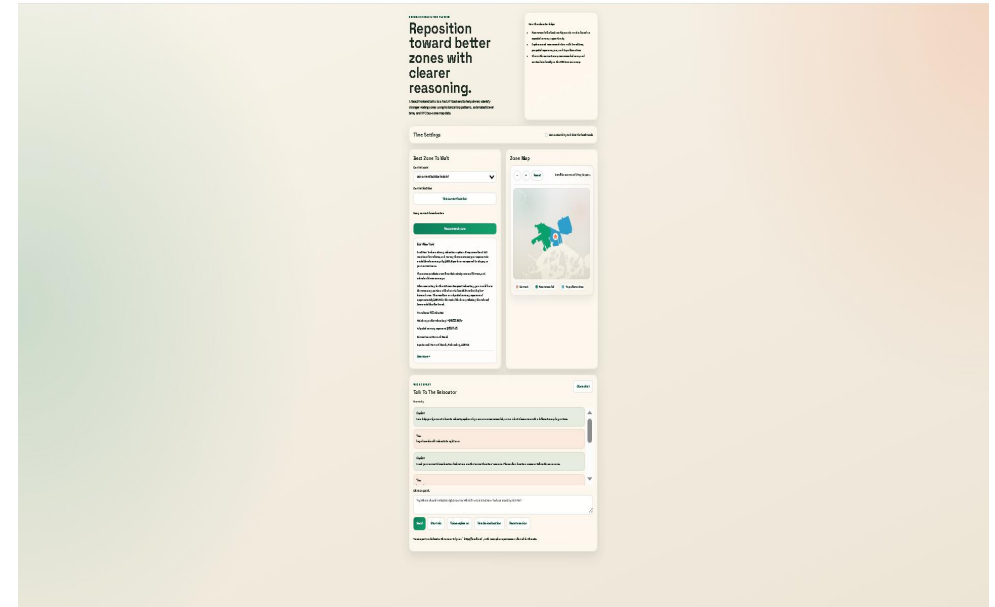
Training rows

0.9749

Avg NDCG

148.6 ms

API response



Live app demo: planner + map + copilot

The decision problem

Drivers need fast, practical relocation guidance between trips.

Problem

- Demand changes by zone, day, and hour.
- Moving zones costs time that could otherwise be spent waiting for requests.
- Newer drivers may not know TLC zone patterns.
- During slow periods, waiting in the wrong zone can waste a large part of a shift.

Solution

- Rank nearby candidate zones instead of giving raw data.
- Account for travel time before recommending relocation.
- Explain the recommendation in plain language.
- Show current, recommended, and alternative zones on a map.

Research Question

How can a machine-learning-powered web app help NYC Uber drivers reduce unproductive waiting time by recommending nearby NYC TLC zones using historical Uber trip patterns, estimated travel time, and adjusted earning exposure?

Outcome: a ranked relocation recommendation, not a guaranteed personal earnings prediction.

Who the app is built for

Two driver personas shaped the interface and feature priorities.

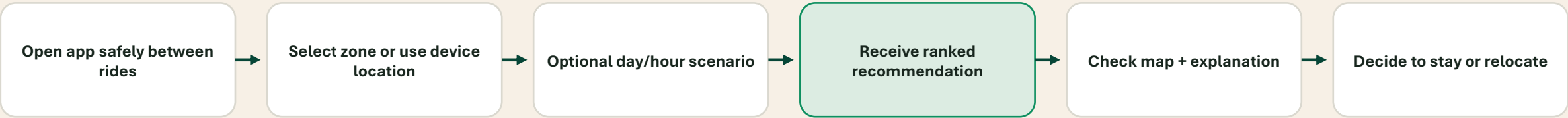
Full-time NYC driver

- Long shifts with changing demand patterns
- Needs quick decisions during slow periods
- Wants an explanation before spending time relocating

Part-time or newer driver

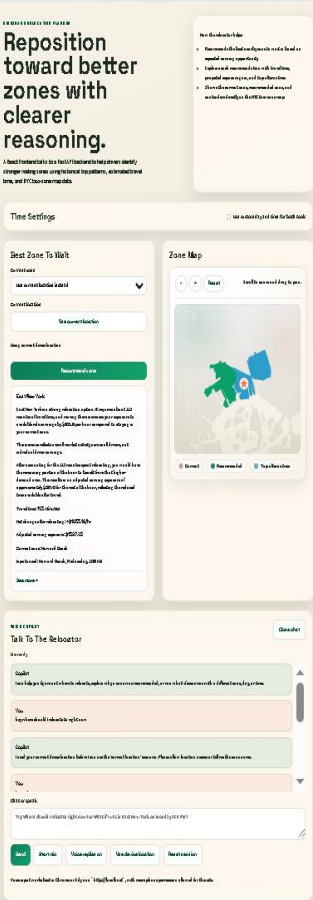
- Less intuition about TLC taxi zones
- Needs map context and simple guidance
- Benefits from natural-language what-if scenarios

User journey



Application overview

Planner, map, recommendation text, alternatives, and copilot are part of one decision-support workflow.



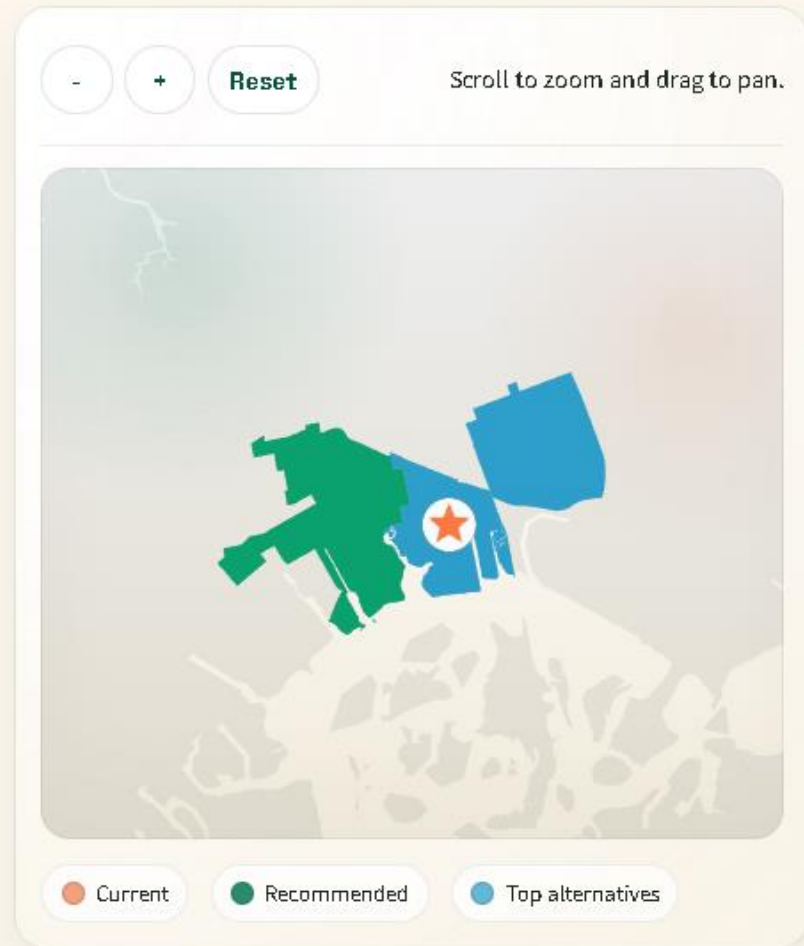
Key capabilities

- Current zone dropdown or device location
- Custom day/hour what-if scenarios
- Recommended zone with travel-time tradeoff
- Top alternatives for comparison
- Taxi-zone map highlights
- Chat/voice copilot explanations

Actual zone-map render

The map gives spatial context for current, recommended, and alternative TLC taxi zones.

Zone Map



Why the map matters

- TLC zone names do not always tell drivers where the area is geographically.
- Color coding separates current zone, recommendation, and alternatives.
- Drivers can compare zones before spending time relocating.
- Future map improvements can add streets and landmarks.

Map legend behavior

- Current zone ● Recommended zone ● Top alternatives

Howard Beach recommendation example

This example ranks East New York first, then South Ozone Park, then staying in Howard Beach.

Recommend zone

East New York

East New York is a strong relocation option. It requires about 11.5 minutes of travel time, and moving there increases your exposure to available ride earnings by \$1055.16 per hour compared to staying in your current zone.

This increase reflects overall market activity across all drivers, not individual driver earnings.

After accounting for the 11.5 minutes spent relocating, you would have the remaining portion of the hour to benefit from that higher-demand area. This results in an adjusted earning exposure of approximately \$1597.45 for the rest of the hour, reflecting the reduced time available after travel.

Travel time: 11.5 minutes

Net change after relocating: +\$1055.16/hr

Adjusted earning exposure: \$1597.45

Current zone: Howard Beach

Inputs used: Howard Beach, Wednesday, 3:00 PM

See more +

Inputs used: Howard Beach, Wednesday, 3:00 PM

See more -

Top options right now:

• East New York

Travel time: 11.5 minutes

Net change after relocating: +\$1055.16/hr

Adjusted earning exposure: \$1597.45

East New York is a strong relocation option. It requires about 11.5 minutes of travel time, and moving there increases your exposure to available ride earnings by \$1055.16 per hour compared to staying in your current zone.

This increase reflects overall market activity across all drivers, not individual driver earnings.

After accounting for the 11.5 minutes spent relocating, you would have the remaining portion of the hour to benefit from that higher-demand area. This results in an adjusted earning exposure of approximately \$1597.45 for the rest of the hour, reflecting the reduced time available after travel.

• South Ozone Park

Travel time: 12.8 minutes

Net change after relocating: +\$488.00/hr

Adjusted earning exposure: \$805.2

South Ozone Park is a strong relocation option. It requires about 12.8 minutes of travel time, and moving there increases your exposure to available ride earnings by \$488.00 per hour compared to staying in your current zone.

This increase reflects overall market activity across all drivers, not individual driver earnings.

After accounting for the 12.8 minutes spent relocating, you would have the remaining portion of the hour to benefit from that higher-demand area. This results in an adjusted earning exposure of approximately \$805.20 for the rest of the hour, reflecting the reduced time available after travel.

• Howard Beach

Travel time: 0 minutes

Net change after relocating: \$0.00/hr

Adjusted earning exposure: \$391.11

Howard Beach is your current zone, so staying here requires no travel time.

This reflects overall market activity across all drivers, not individual driver earnings.

Your adjusted earning exposure for the rest of the hour remains approximately \$391.11.

Ranking output

1

East New York

11.5 min • +\$1055.16/hr

2

South Ozone Park

12.8 min • +\$488.00/hr

3

Howard Beach

0.0 min • \$0.00/hr

The stay-put option is included so the model can recommend not relocating when moving is not worth it.

Voice/text copilot

The copilot reduces reading load and supports natural-language what-if scenarios.

VOICE COPILOT

Close chat

Talk To The Relocator

Mic ready

Copilot

I can help you figure out where to relocate, explain why a zone was recommended, or run what-if scenarios with a different zone, day, or time.

You

hey where should I relocate to right now

Copilot

I need your current device location before I can use the 'current location' scenario. Please allow location access or tell me the zone name.

You

Chat or speak

Try: Where should I relocate right now? or What if I was in East New York on Monday at 5 PM?

Send

Start mic

Voice replies on

Use device location

Reset session

What it adds

- Driver can ask “where should I relocate right now?”
- Supports what-if questions by zone, day, and time.
- Explains model recommendations in plain language.
- Works as a convenience layer over the recommendation service.

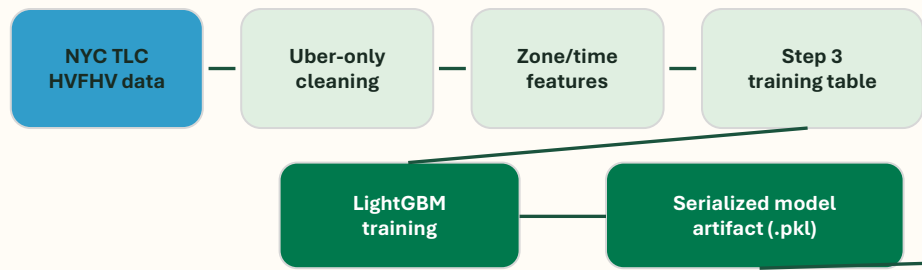
Design reason

User feedback suggested that some drivers would rather ask questions naturally than read every field manually.

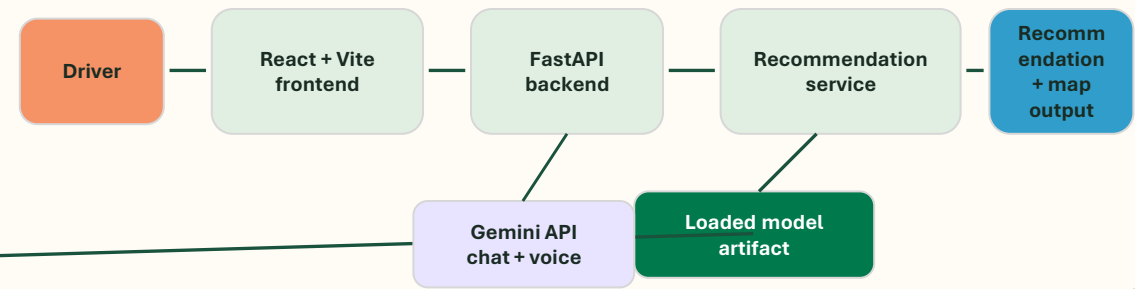
End-to-end website flow and model pipeline

Solid arrows = implemented app path. Dashed arrows = planned future data extensions.

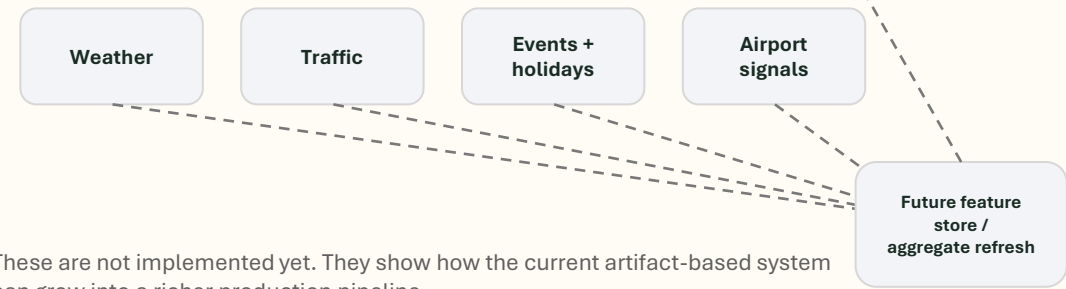
1. Offline data + model build



2. Live website recommendation path



3. Future data extensions



These are not implemented yet. They show how the current artifact-based system can grow into a richer production pipeline.

Why this structure?

- React keeps the driver interface fast and interactive.
- FastAPI exposes clean endpoints for recommendations, zones, GeoJSON, and copilot requests.
- The recommendation service separates ranking logic from the UI.
- The LightGBM model is saved as a reusable artifact, so it can be loaded, versioned, or replaced.
- Future signals can be added to the feature pipeline without rebuilding the entire website.

Why these frameworks and tools?

Each tool was selected to support fast iteration, model serving, and a demo-ready user interface.

React + Vite

Fast interactive UI, component-based screens, quick local builds, and simple deployment path for a web demo.

FastAPI

Python-native backend, strong fit for ML serving, automatic API structure, and low-latency JSON endpoints.

LightGBM

Efficient gradient boosting for millions of tabular rows with nonlinear zone/time interactions.

GeoJSON + zone lookup

Lets the app show real TLC taxi-zone geography instead of only text recommendations.

Gemini API

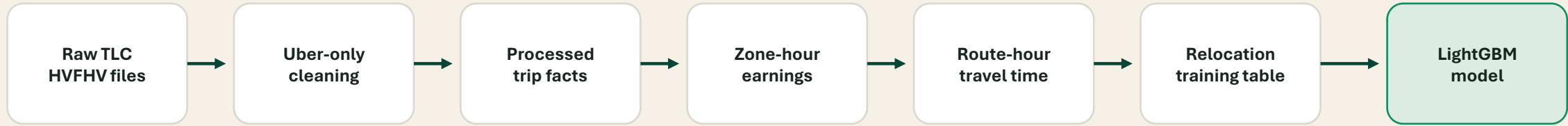
Adds natural-language and voice support without replacing the deterministic recommendation endpoint.

PostgreSQL/RDS future path

Production version can store refreshed aggregates, model registry data, and monthly source lineage.

Data pipeline and feature construction

Historical trips are transformed into zone/time and route/time features used for ranking.



Core engineered tables

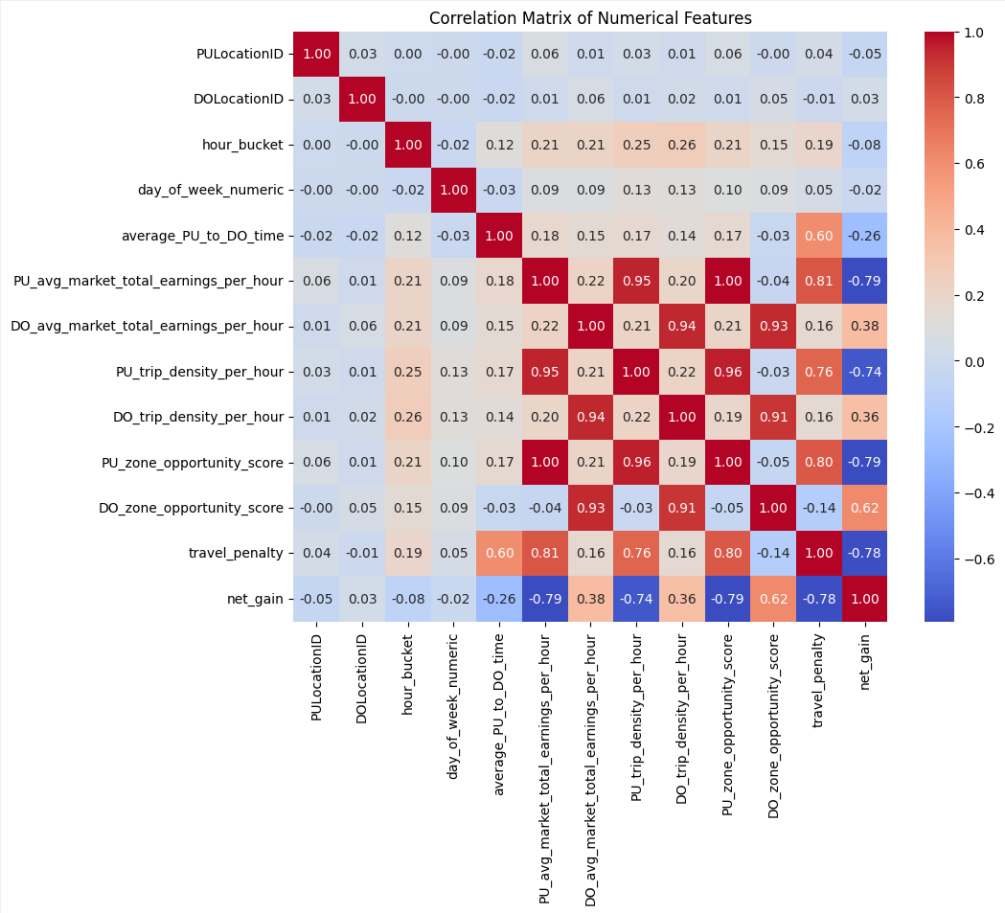
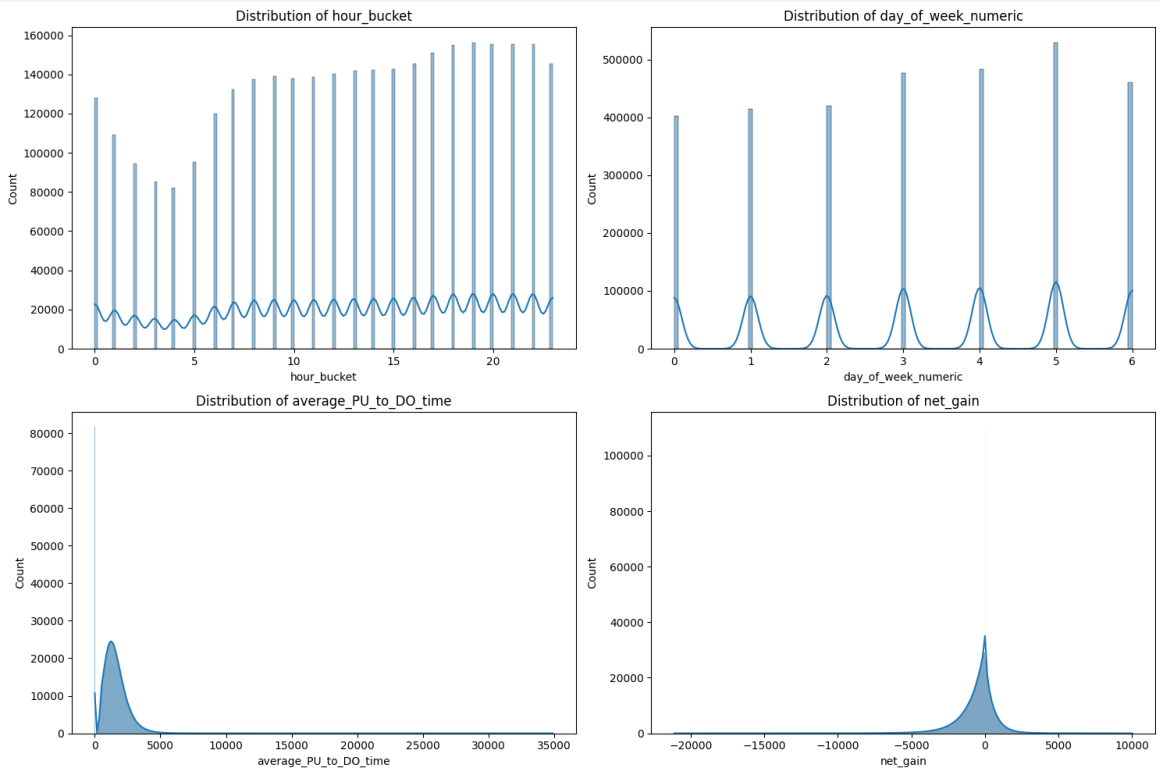
- Zone/day/hour earning opportunity metrics
- Route/day/hour average travel-time metrics
- Candidate relocation rows with current zone, destination zone, hour, day, and travel time

Why aggregate first?

- Recommendations should not scan raw trip rows at runtime.
- Aggregates make model input and explanation fields fast to look up.
- The same design maps cleanly to future PostgreSQL/RDS tables.

Exploratory data analysis

EDA verified feature distributions and helped identify relationships before modeling.



EDA confirmed the training table includes time patterns, travel-time variation, and strong relationships among earning, density, opportunity score, travel penalty, and net_gain.

Model formulation: recommendation as regression ranking

The app needs a ranked list of candidate zones, not just a yes/no relocation label.

Input context

- Current zone
- Candidate destination zone
- Hour of day
- Day of week
- Average travel time

Model output

predicted_net_gain

A continuous score representing the historical relocation benefit after accounting for travel-time cost.

Higher score = stronger relocation candidate

Why regression?

- A binary classifier would only say relocate or not.
- A numeric score allows many zones to be compared and sorted.
- The backend can include stay-put as a candidate.
- The user receives top alternatives, not a single blind answer.

Why LightGBM Regressor was selected

LightGBM matches the project's data shape: large, tabular, nonlinear, and ranking-oriented.

Compared with simpler models

- Linear regression is easier to explain but too limited for zone × hour × day interactions.
- Rule-based scoring is transparent but cannot learn complex patterns from millions of rows.
- Deep learning is more complex than necessary for this structured tabular data.

Why gradient-boosted trees

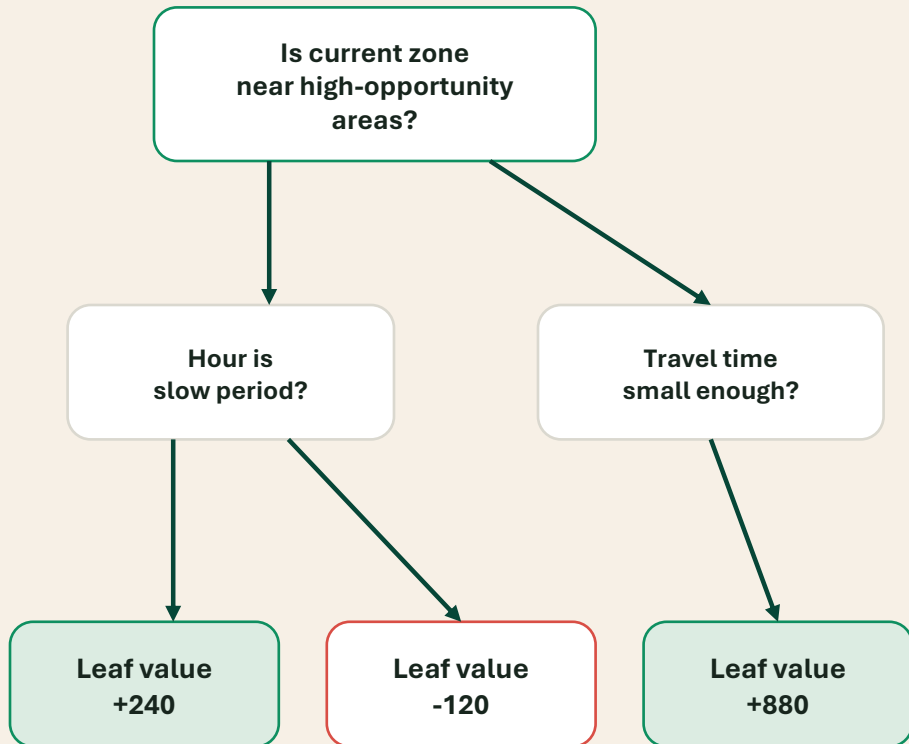
- Decision trees split data into regions such as zone/time/travel-time combinations.
- Boosting adds trees sequentially, where each tree corrects errors left by earlier trees.
- This captures nonlinear patterns without manually coding every interaction.

Why LightGBM specifically

- Fast training on millions of rows
- Efficient prediction for API serving
- Handles categorical-like numeric zone IDs through learned splits
- Provides feature importance for explanation and debugging

How a tree model outputs a continuous value

A regression tree does not vote for a class; each leaf stores a numeric prediction.



Gradient boosting prediction

Each tree outputs a number. LightGBM combines many trees by adding their outputs together.

$$\hat{y} = \text{initial average} + \text{tree}_1(x) + \text{tree}_2(x) + \dots + \text{tree}_n(x)$$

Because the leaf values are numeric, the final prediction is continuous. That is how a tree-based model can predict a dollar-like net_gain score rather than only a category.

In this app, the continuous score is used to rank zones.

How LightGBM handles nonlinear relationships

The value of a relocation zone can depend on combinations of current zone, candidate zone, day, hour, and travel time.

Nonlinear example

A 12-minute relocation may be worth it at 3 PM from one zone, but not worth it at 11 PM from another zone.

- Trees split the feature space into many rule-like regions.
- Boosting combines many trees, allowing curved and interaction-heavy decision surfaces.
- The model can learn that the same travel time has different meaning depending on origin, destination, and hour.

Why this matters for rideshare

- Demand is spatial: some zones are stronger than others.
- Demand is temporal: patterns change by hour and day.
- Travel time is a cost: longer repositioning reduces remaining opportunity.
- The recommendation must combine these factors rather than treating them independently.

Target variable and loss function

The model learns to predict `net_gain` and is penalized more for large numeric mistakes.

Target construction

**`net_gain` = destination earning exposure –
current-zone earning exposure – travel penalty**

- Positive `net_gain` means the candidate zone historically looked stronger after relocation cost.
- Negative `net_gain` means travel time or weaker destination opportunity made the move less attractive.
- This target supports ranking because each candidate receives a numeric score.

Squared-error style regression loss

**Loss = average((actual `net_gain` – predicted
`net_gain`)²)**

- Small errors receive small penalties.
- Large errors receive much larger penalties.
- This is useful because sending a driver toward a very weak zone is more harmful than a small ordering mistake.
- RMSE is aligned with this squared-error training objective.

Hyperparameter tuning with Optuna

Optuna searched LightGBM settings and selected the configuration with the lowest validation RMSE.

What was tuned

n_estimators	700
learning_rate	0.0383
num_leaves	70
max_depth	11
min_child_samples	40
subsample	0.8
colsample_bytree	0.9
reg_alpha	0.0519
reg_lambda	0.00543

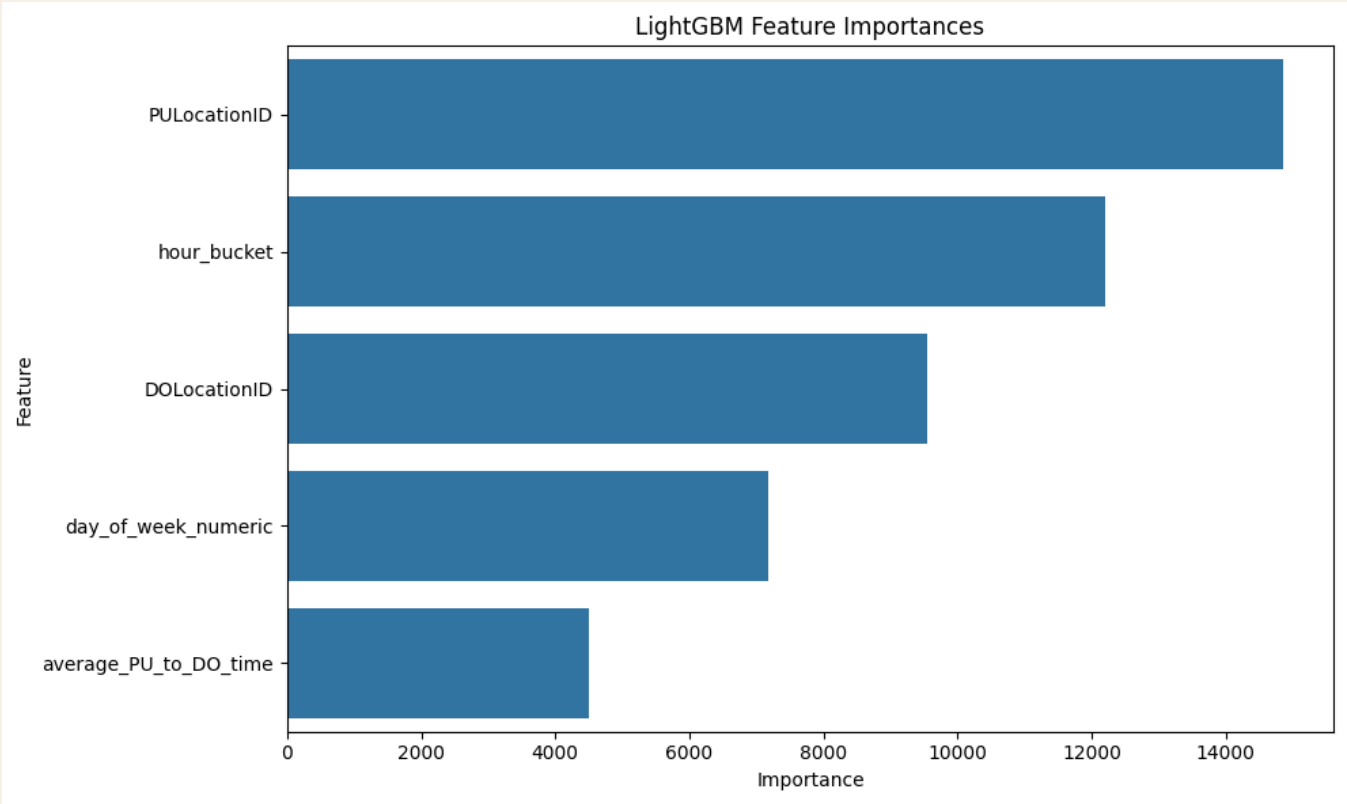
Why tuning matters

- Too few trees can underfit and miss demand patterns.
- Too many or too-deep trees can overfit historical noise.
- Learning rate controls how aggressively each tree updates the model.
- Sampling and regularization reduce overfitting.
- RMSE was minimized because large score errors are risky for recommendations.

Tuning objective: minimize RMSE

Feature importance

The model relied most on current zone, hour, candidate zone, day, and travel time.



Interpretation

- PULocationID is strongest because the current zone determines the starting opportunity and candidate set.
- hour_bucket is important because rideshare opportunity varies heavily by time of day.
- DOLocationID matters because the destination zone defines the relocation opportunity.
- average_PU_to_DO_time matters because longer travel reduces the remaining usable hour.

Model performance: what the numbers mean

The metrics evaluate both numeric prediction error and recommendation ranking quality.

MAE

317.4087

Average absolute size of prediction error in net_gain units. Lower is better.

RMSE

460.4430

Penalizes large mistakes more than MAE. Useful because very wrong relocation scores are costly.

R²

0.8854

Share of target variation explained by the model. Indicates strong fit to historical net_gain patterns.

NDCG

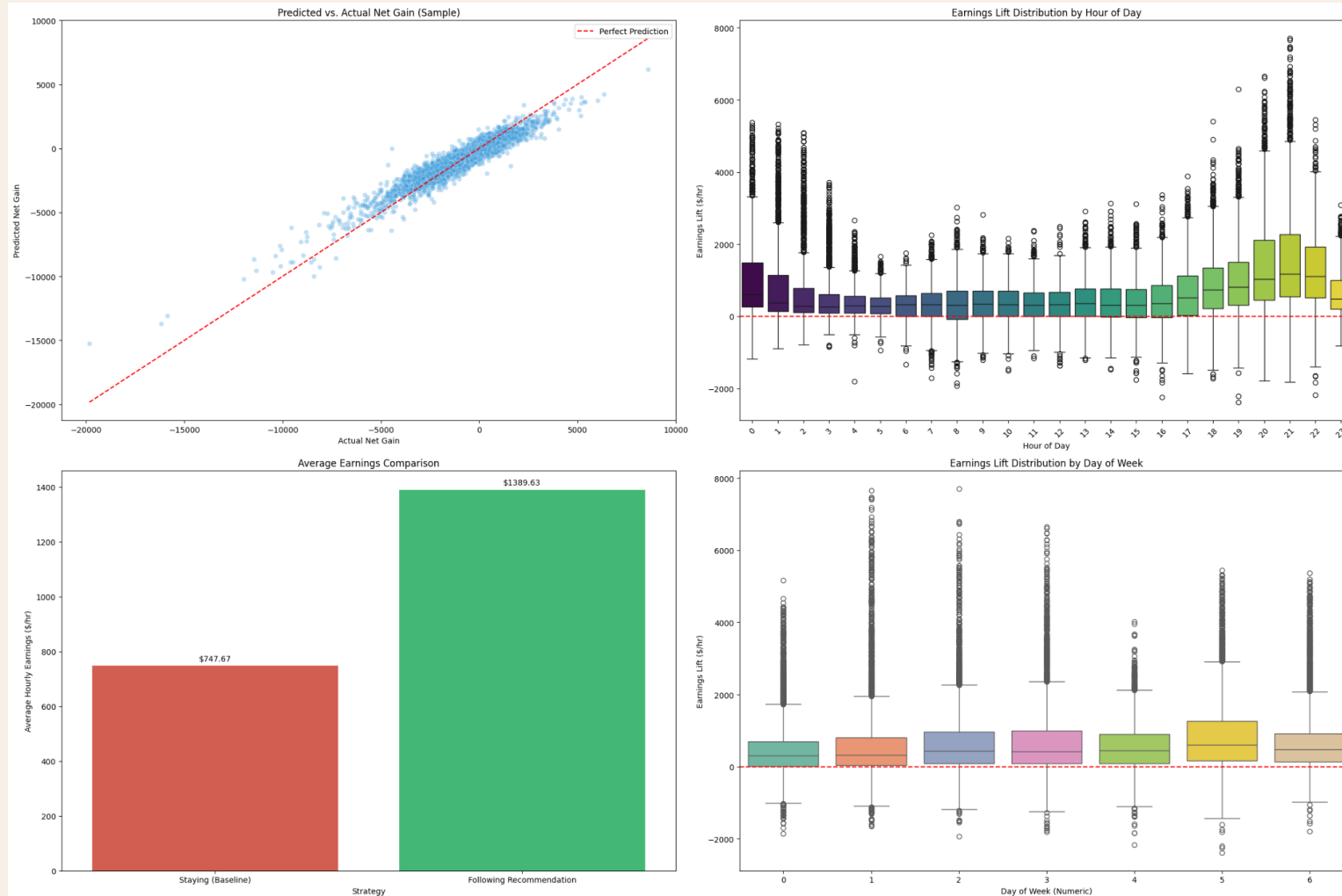
0.9749

Ranking metric showing whether the strongest relocation candidates appear near the top. Crucial for recommender behavior.

Why both error and ranking metrics? The app shows a ranked list of zones. Numeric error matters, but ordering better zones above weaker zones is directly tied to user experience.

Model evaluation dashboard

The notebook evaluated prediction fit and historical recommendation impact across time patterns.

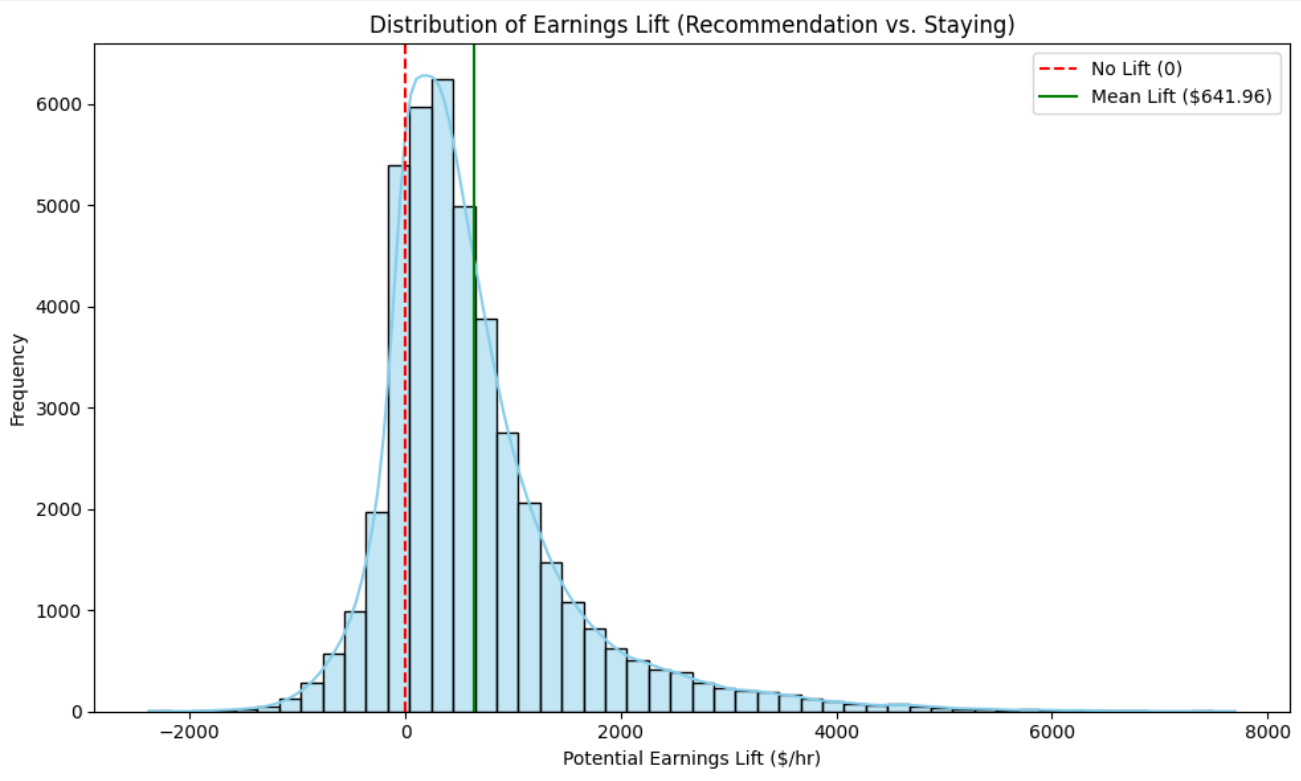


Dashboard views

- Predicted vs. actual net_gain checks whether predictions follow actual values.
- Earnings lift by hour checks when recommendations show more benefit.
- Average staying vs. following recommendation gives a high-level impact comparison.
- Day-of-week distribution checks whether lift patterns are consistent across days.

Historical recommendation impact

The notebook compared staying put against following the model recommendation where a stay-put baseline existed.



Simulation results

42,367

Simulated contexts

\$747.67/hr

Stay-put exposure

\$1,389.63/hr

Recommendation exposure

+\$641.96/hr

Mean lift

Interpretation

The model often found zones with stronger historical market earning exposure than staying put. This is not guaranteed individual driver pay; it is a historical market-opportunity signal.

Technical testing and user feedback

Testing confirmed the app flow worked locally and user feedback changed the final product scope.

Manual technical testing

- Backend health endpoint returned status OK.
- Zone dropdown populated with TLC zone names and IDs.
- Recommendation endpoint returned ranked result in 148.6 ms locally.
- GeoJSON map rendered 263 taxi-zone features.
- Copilot worked when Gemini API key/quota was valid.
- Errors were shown in the UI instead of failing silently.

User-driven iterations

- Removed offered-trip prediction because a part-time driver was concerned about rejecting rides affecting ratings/perks.
- Kept explanation text because a full-time driver found the recommendation aligned with local driving intuition.
- Added chat/voice copilot because users wanted a lower-reading interaction style.
- Kept manual zone selection because browser geolocation can be imperfect.

Limitations and next steps

The current app is a strong prototype; production improvements would add live context and repeatable data refresh.

Current limitations

- Historical patterns do not guarantee future individual earnings.
- No live weather, events, traffic, holidays, or driver supply signals yet.
- Some zone/time combinations may have limited data.
- Device location depends on browser permission and accuracy.

Next production improvements

- Monthly data refresh and rolling averages
- PostgreSQL/RDS tables for aggregates and model registry
- Weather, event, airport, and holiday features
- Street/landmark map layer for better driver recognition
- Live driver testing to measure wait-time reduction

Driver Earnings Navigator

Smart relocation. Better reasoning. Clearer driver decisions.

Project takeaway

This capstone combines machine learning, geospatial visualization, backend model serving, frontend UX, and conversational AI into a practical decision-support app for NYC Uber drivers.

Historical market exposure, not guaranteed individual earnings.